

TEKNİK RAPOR

Spotify Büyük Veri Pipeline Projesi

Ders	Büyük Veri Analizine Giriş
Öğretim Üyesi	Dr. Ayşe Gül Eker
Veri Seti	Spotify Tracks Dataset — 114.000 şarkı, 113 müzik türü
Problem Türü	Sınıflandırma — track_genre tahmini (çok sınıflı)
Tarih	Mayıs 2026

İsim	Numara	Görev
Sabri Eren Yeşildağ	220201013	Docker kurulumu + Kafka Producer/Consumer
Kezban Ceylin Atlı	220201047	Spark Structured Streaming + Delta Lake
Zehra Karabektaş	220201121	EDA + Feature Engineering
Tolga Ferhan Küçük	220201009	ML Modelleri + MLflow + Dashboard

BÖLÜM 1: PROJENİN AMACI VE KAPSAMI

1.1 Projenin Genel Hedefi

Bu proje, Spotify müzik platformuna ait 114.000 şarkıdan oluşan veri seti üzerinde uçtan uca bir büyük veri pipeline'ı geliştirmeyi hedeflemektedir. Temel amaç; Docker, Kafka, Spark, Delta Lake ve MLflow teknolojilerini bir arada kullanarak şarkıların müzik özelliklerinden hareketle türlerini (track_genre) tahmin etmektir.

1.2 Veri Setinin Seçilme Gerekçesi

Spotify Tracks Dataset (Kaggle), 114.000 şarkı ve 20 kolon içermesiyle makine öğrenmesi uygulamaları için zengin bir kaynak sunmaktadır. Danceability, energy, tempo ve loudness gibi sayısal özellikler ile track_genre gibi kategorik değişkenler, veri temizleme ve sınıflandırma problemleri için ideal bir yapı oluşturmaktadır. 113 farklı müzik türü içermesi, çok sınıflı sınıflandırma için gerçekçi bir senaryo sağlamaktadır.

BÖLÜM 2: SİSTEM MİMARİSİ

2.1 Genel Katman Yapısı

Proje altı temel katmandan oluşmaktadır. Docker ile tüm servisler izole konteynerler içinde çalıştırılmakta, Kafka ile gerçek zamanlı veri akışı simüle edilmekte, Spark Structured Streaming ile veri işlenmekte, Delta Lake ile depolanmakta, ML modelleri MLflow ile takip edilmekte ve Matplotlib/Seaborn ile görselleştirilmektedir.

Katman	Teknoloji	Açıklama
Konteynerizasyon	Docker + Compose	6 servis — Zookeeper, Kafka, Spark (x2), Producer, MLflow

Katman	Teknoloji	Açıklama
Veri Üretimi	Kafka Producer	500 mesaj/sn — 23 alanlı JSON — spotify-tracks topic
Veri İşleme	Spark 3.5.0	Structured Streaming — awaitTermination(480sn) — trigger(30sn)
Depolama	Delta Lake	Bronze / Silver / Gold madalyon mimarisi
ML ve Takip	MLflow 2.11.0	5 model — parametreler, metrikler ve modeller loglandı
Görselleştirme	Matplotlib + Seaborn	11 dashboard görseli — PNG formatında kaydedildi

2.2 Bronze / Silver / Gold Mimarisi

Katman	Satır Sayısı	Açıklama
Bronze (Ham)	114.000	Kafka'dan streaming ile okundu, kafka_timestamp eklendi
Silver (Temiz)	89.740	24.260 kirli kayıt giderildi — null, duplike ve format hataları temizlendi
Gold (Özet)	113 tür	Tür bazında ort. popularity, danceability, energy, tempo, loudness, valence
Gold/features	89.740 satır 28 kolon	Feature Engineering çıktısı — /home/jovyan/delta-lake/gold/features

BÖLÜM 3: DOCKER VE KAFKA AŞAMASI

3.1 Docker Kurulumu

Her servis için ayrı Dockerfile oluşturulmuş ve tüm servisler docker-compose.yml dosyasında tanımlanmıştır. Servisler spotify-net adlı ortak Docker network üzerinden haberleşmektedir. Delta Lake verisi delta-lake-data adlı kalıcı Docker volume üzerinde saklanmaktadır. Spark için jupyter/pyspark-notebook:spark-3.5.0 temel imajı kullanılmış; mlflow==2.11.0, kafka-python==2.0.2, matplotlib==3.7.1, seaborn==0.12.2, pandas==2.1.4, numpy==1.24.3 ve scikit-learn==1.3.0 paketleri Dockerfile üzerinden yüklenmiştir.

3.2 Kafka Producer Yapısı

Python ile yazılan Kafka Producer, dataset.csv dosyasını pandas ile okuyarak her satırı 23 alanlı JSON formatına dönüştürmekte ve spotify-tracks topic'ine göndermektedir. Her mesaj kafka_timestamp, user_id (rastgele UUID), event_type ve 20 müzik özelliği içermektedir. MESSAGES_PER_SECOND=500 ayarı ile 114.000 mesajın tamamı yaklaşık 4 dakikada gönderilmiştir.

3.3 Karşılaşılan Zorluklar ve Çözümler

Zorluk	Çözüm
Kafka ve Delta Lake JAR paketleri eksikti	spark.jars.packages ile spark-sql-kafka ve delta-spark_2.12:3.2.0 JAR eklendi
pip install delta-spark ile JAR paketi çalışmıyordu	pip paketi Dockerfile'dan kaldırıldı, yalnızca JAR kullanıldı
Docker volume izin hatası (root/jovyan sorunu)	Dockerfile'a mkdir ve chown komutları eklenerek izinler düzeltildi
ENV satırı çok satıra bölününce NullPointerException oluştu	ENV tek satır olarak yazıldı

BÖLÜM 4: SPARK VE DELTA LAKE AŞAMASI

4.1 Spark Structured Streaming ve Şema Tanımlaması

Apache Spark 3.5.0 kullanılarak Kafka topic'inden gelen JSON mesajları Spark Structured Streaming ile gerçek zamanlı olarak okunmaktadır. Geliştirme aşamasında önce CSV'den manuel okuma yapılarak pipeline test edilmiş, ardından Kafka entegrasyonuna geçilmiştir. 23 kolonlu şema önceden tanımlanmış olup StringType, IntegerType ve FloatType kullanılmıştır. awaitTermination(timeout=480) ile Producer bitene kadar beklenmekte, trigger(processingTime='30 seconds') ile Delta Lake'e büyük dosyalar yazılarak small files problemi önlenmektedir.

4.2 Veri Temizleme Aşamaları

Aşama	İşlem	Sonuç
Aşama 1	Null temizleme — dropna() tüm kolonlar	114.000 → 113.999 satır (1 satır silindi)
Aşama 2	Duplike temizleme — dropDuplicates(['track_id'])	113.999 → 89.740 satır (24.259 duplike silindi)
Aşama 3	Popularity filtresi — 0-100 aralığı kontrolü	89.740 satır (hiçbir kayıt dışarıda kalmadı)
Aşama 4	explicit dönüşümü — 'True'/'False' → 1/0	Format düzeltildi — when() kullanıldı

Not: Aşama 2 ve 3 sonuçlarının aynı olması normaldir. Aynı şarkı farklı türlerde listelenebilmekte olup dropDuplicates her track_id için yalnızca bir kayıt tutar.

4.3 Karşılaşılan Zorluklar ve Çözümler

Geliştirme sürecinde Kafka entegrasyonunda birden fazla kritik sorunla karşılaşılmıştır. 'Failed to find data source: kafka' hatası spark.jars.packages yapılandırması ile çözülmüştür. Delta Lake JAR versiyonu uyumsuzluğundan kaynaklanan NoSuchElementException hatası pip paketinin kaldırılmasıyla giderilmiştir. Checkpoint klasörü oluşturulamaması ve Docker volume izin sorunları Dockerfile düzenlemeleriyle çözülmüştür. CSV ve Kafka karşılaştırması 89.740 satırda eşleşip pipeline doğruluğu teyit edilmiştir.

BÖLÜM 5: KEŞİFSEL VERİ ANALİZİ (EDA)

5.1 Temel İstatistikler

Ölçüt	Değer
Toplam satır (Silver katmanı)	89.740
Toplam kolon	23
Farklı müzik türü sayısı	113
Popularity ortalaması	33,2 (min: 0, max: 100)
En popüler 3 tür	k-pop, pop-film, metal
Null değer durumu	Hiçbir kolonda null değer tespit edilmedi
Tempo anomalisi	157 kayıтта tempo=0 (%0,17) — model performansına etkisi ihmal edilebilir

5.2 Zaman Serisi Analizi

Tüm veri saat 09:00'da tek bir Kafka oturumunda aktarılmıştır. Dakika bazında incelendiğinde 15-21. dakikalar en yoğun dilim olup yaklaşık 19.000 mesaj/dakika ile en yüksek hıza ulaşılmıştır. Pipeline'ın kesintisiz ve kararlı çalıştığı teyit edilmiştir. Tüm verinin aynı kafka_timestamp değerine sahip olması nedeniyle anlamlı bir zaman serisi trendi elde edilememiştir.

5.3 En Önemli EDA Bulguları

Bulgu	Açıklama
Hip-hop ve speechiness	Hip-hop, speechiness değeriyle tüm türlerden belirgin şekilde ayrışıyor — güçlü ayırt edici özellik.
Energy-Loudness korelasyonu	$r=0,76$ ile çok yüksek korelasyon. Popularity ise hiçbir değişkenle güçlü korelasyon göstermiyor.
Popularity dağılımı	Sola yığılmış: şarkıların büyük çoğunluğu 20-29 aralığında. Explicit şarkılar daha popüler (36,9 vs 32,9).
Tempo bimodal dağılım	120-130 BPM civarında çift tepeli dağılım — farklı türlerin tempo alışkanlığını yansıtıyor.
Tür dengesizliği	Reggaeton (72), indie (134) gibi türler çok az temsil ediliyor — ML aşamasında ek önlem gerektirdi.

5.4 Karşılaşılan Zorluklar

EDA aşamasında iki temel zorlukla karşılaşılmıştır. speechiness, instrumentality ve loudness kolonlarında güçlü aykırı değerler tespit edildi. Tüm verinin aynı kafka_timestamp değerine sahip olması nedeniyle anlamlı bir zaman serisi trendi elde edilemedi; event_type kolonunun da yalnızca 'track_played' değerini içermesi bu kolonların analizde kullanılamamasına yol açtı.

BÖLÜM 6: ÖZELLİK MÜHENDİSLİĞİ (FEATURE ENGINEERING)

6.1 Üretilen Özellikler

Özellik Adı	Formül	Açıklama
energy_danceability_ratio	$\text{energy} / (\text{danceability} + 0,0001)$	Enerjik ama az dans edilebilir şarkıları ayırt eder. En yüksek: sleep türü.
loudness_normalized	$(\text{loudness} - \text{min}) / (\text{max} - \text{min})$	Loudness -49,53/+4,53 dB aralığını 0-1'e normalize eder. Ortalama: 0,759.
tempo_category	yavaş (<100 BPM) orta (100-140 BPM) hızlı (>140 BPM)	Şarkıların yarısından fazlası orta grupta. ML için kategorik özellik.
energy_acoustic_contrast	$\text{energy} - \text{acousticness}$	Yüksek değer: metal/rock/grindcore. Düşük değer: sleep/piano/classical. RF önem sıralamasında orta-yüksek katkı sağlamıştır.
dancefloor_score	$(\text{danceability} + \text{valence}) / 2$	Dans edilebilirlik ve pozitif duygu tonunu birleştirir. En yüksek: children/kids. En düşük: sleep, show-tunes, romance.

Özellikler Delta Lake Gold katmanına (/home/jovyan/delta-lake/gold/features) 89.740 satır ve 28 kolon olarak yazılmış ve doğrulanmıştır.

6.2 Karşılaşılan Zorluklar ve Çözümler

Sıfıra bölme hatası: energy_danceability_ratio hesaplanırken danceability=0 olduğu durumlarda paydaya 0,0001 eklendi. Ayrıca speechiness, instrumentality ve loudness'te güçlü aykırı değerler tespit edildi; loudness normalizasyonla 0-1 arasına çekildi.

BÖLÜM 7: MAKİNE ÖĞRENMESİ VE MLFLOW

7.1 Model Eğitimi ve Sonuçları

PySpark MLlib kullanılarak 5 farklı sınıflandırma modeli eğitilmiştir. Her model için MLflow'da ayrı experiment ve run oluşturulmuş; parametreler, metrikler ve model dosyaları loglanmıştır. track_genre sınıflandırması gerçekleştirilmiştir.

Model	Accuracy	F1-Score	AUC-ROC	Durum
Logistic Regression	0,466	0,449	0,920	
Decision Tree Classifier	0,481	0,480	0,899	
Random Forest Classifier	0,560	0,547	0,944	EN İYİ
GBT Classifier	0,506	0,503	0,928	
Naive Bayes	0,299	0,256	0,855	

7.2 Özellik Önem Sıralaması (Feature Importance)

Özellik	Önem Skoru	Düzy
popularity	0,2417	En yüksek etki
instrumentalness	0,0925	İkinci sıra
danceability	0,0909	Üçüncü sıra
acousticness	0,0899	Dördüncü sıra
speechiness	0,0875	Beşinci sıra
dancefloor_score	0,0873	Altıncı sıra
energy_acoustic_contrast	0,0800	Yedinci sıra
key	0,0049	Düşük etki
mode	0,0033	Çok düşük etki
time_signature	0,0027	En düşük etki

7.3 Karışıklık Matrisi (Confusion Matrix) Analizi

Karışıklık matrisi incelendiğinde yakın müzikal özelliklere sahip türlerin birbirine karıştığı görülmektedir. En çok karışan türler: breakbeat → chicago-house, sleep → iranian, idm → study, malay → cantopop, heavy-metal → black-metal. Bu karışmaların müzikal özellikler açısından benzer türler arasında gerçekleşmesi beklenen bir sonuçtur.

7.4 MLflow Entegrasyonu

Her model için ayrı MLflow experiment ve run oluşturulmuştur. mlflow.log_param() ile tüm parametreler, mlflow.log_metric() ile tüm metrikler loglanmış, mlflow.spark.log_model() ile model dosyaları kaydedilmiştir. MLflow kullanıcı arayüzü <http://localhost:5000> adresinde görüntülenebilmektedir.

7.5 Karşılaşılan Zorluklar ve Çözümler

Zorluk	Çözüm
Spark bellek hatası (113 tür — yüksek bellek kullanımı)	Model parametreleri hafifletildi, veri örnekleme yapıldı
Timestamp kolon adı uyumsuzluğu	kafka_timestamp ve timestamp kolonlarının ikisi de desteklendi
MLflow artifact kaydında /mlflow izin hatası oluştu	MLflow Dockerfile içinde --serve-artifacts ve --artifacts-destination /mlflow/artifacts ayarları eklendi

BÖLÜM 8: SONUÇLAR VE DEĞERLENDİRME

8.1 Proje Özeti

Ölçüt	Değer
İşlenen toplam şarkı (Bronze)	114.000
Temizlenmiş kayıt (Silver)	89.740
Silinen kirli kayıt	24.260
Müzik türü sayısı (Gold)	113
En iyi model	Random Forest Classifier
Seçim metriği (f1_weighted)	0,5469
En iyi AUC-ROC değeri	0,944
Dashboard görseli sayısı	11 PNG dosyası

8.2 Öğrenilen Dersler

- Büyük veri teknolojileri birbirine entegre çalışır — Docker olmadan Kafka, Kafka olmadan Spark düşünülemez.
- Özellik mühendisliği ML modellerinin başarısını doğrudan etkiler. En etkili özellikler popularity, instrumentality ve danceability olarak belirlenmiştir.
- MLflow ile deney takibi yapılmadan 5 modeli karşılaştırmak çok daha zor olurdu.
- Delta Lake Bronze/Silver/Gold mimarisi veri güvenilirliğini ve tekrar edilebilirliği artırır.
- Statik bir veri setini Kafka'dan geçirmek gerçek bir akış senaryosu oluşturmaz. Gerçek akış için verinin zaman içinde üretilmesi gerekir.
- Büyük veri ortamında bellek yönetimi kritiktir — model parametrelerini dikkatli ayarlamak gerekir.
- EDA'da görsel analiz çok önemlidir: korelasyon ısı haritası ve dağılım grafiği olmadan güçlü bağlar gözden kaçabilirdi.